```
// FILENAME: MainForm.cs
// WRITTEN BY: Tyler J. Millershaski
// DATE CREATED: 06 Sep 2026
//
// PART OF PROJECT: ChargeEm
//
// FILE PURPOSE:
//  This file contains the main form and its associated processing logic.
//
// CLASS NAME: MainForm
//
// CLASS PURPOSE:
//  Handle the input and output of the insurance quotation form. The designer
//  partial class supplies the visual controls; this file contains their event
//  handling, calculation helpers, display formatting, and input-warning behavior.
//
// CLASS CONSTANT DICTIONARY (in Alphabetical Order):
//  SALES_TAX_RATE (double) - The 0.06 tax multiplier used by this assignment,
//                            applied to the premium after discount.
//
// CLASS VARIABLE DICTIONARY (in Alphabetical Order):
//  (None declared in this file.) The form's control fields are supplied by the
//  designer partial class. The class constant is documented separately above.
//
// MODIFICATION HISTORY:
// WHO              WHEN         WHAT
// Millershaski   06 Sep 2026  Initial Version
// --- -------- ------------------------------------------------
namespace ChargeEm
{
    public partial class MainForm : Form
    {
        const double SALES_TAX_RATE = 0.06; // This is the rate for Michigan


        // METHOD NAME: MainForm (CONSTRUCTOR)
        // WRITTEN BY: Tyler J. Millershaski
        // DATE CREATED: 06 Sep 2026
        //
        // METHOD PURPOSE:
        //  Construct the main form and initialize the controls and layout defined in the
```

```
42          //    designer partial class.
43          //
44          // PARAMETERS LIST (in Parameter Order):
45          //  (None)
46          //
47          // RETURNS:
48          //  (Nothing)
49          //
50          // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
51          //  (None)
52          //
53          // MODIFICATION HISTORY:
54          // WHO          WHEN            WHAT
55          // Millershaski 06 Sep 2026     Initial Version
56          public MainForm()
57          {
58              InitializeComponent();
59          }
60
61
62
63          // METHOD NAME: OnLoad
64          // WRITTEN BY: Tyler J. Millershaski
65          // DATE CREATED: 06 Sep 2026
66          //
67          // METHOD PURPOSE:
68          //  Run the inherited form-load behavior, then connect each numeric input text box to
69          //    the handler that clears its warning color when the user types.
70          //
71          // PARAMETERS LIST (in Parameter Order):
72          //  e (EventArgs) - Event information supplied when the form loads.
73          //
74          // RETURNS:
75          //  (Nothing; void.)
76          //
77          // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
78          //  (None)
79          //
80          // MODIFICATION HISTORY:
81          // WHO          WHEN            WHAT
82          // Millershaski 06 Sep 2026     Initial Version
83          protected override void OnLoad(EventArgs e)
84          {
85              base.OnLoad(e);
86
87              // Subscribe to the KeyPress event for each TextBox to reset
```

```
                the warning when the user starts typing
            txtAge.KeyPress += ResetWarning;
            txtHeight.KeyPress += ResetWarning;
            txtWeight.KeyPress += ResetWarning;
            txtCoverageAmount.KeyPress += ResetWarning;
            txtPercentageDiscount.KeyPress += ResetWarning;
            txtFlatDiscount.KeyPress += ResetWarning;
        }



        // METHOD NAME: ResetWarning
        // WRITTEN BY: Tyler J. Millershaski
        // DATE CREATED: 06 Sep 2026
        //
        // METHOD PURPOSE:
        //  Restore the sending text box to a white background when it raises a KeyPress event.
        //    This clears the visual warning without validating the new contents.
        //
        // PARAMETERS LIST (in Parameter Order):
        //  sender (object?) - The control that raised the event. Null or non-TextBox senders are ignored.
        //  e (KeyPressEventArgs) - Information about the keypress (not used)
        //
        // RETURNS:
        //  (Nothing; void.)
        //
        // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
        //  textBox (TextBox) - The sender after the pattern match confirms that it is a textbox.
        //
        // MODIFICATION HISTORY:
        // WHO          WHEN            WHAT
        // Millershaski 06 Sep 2026     Initial Version
        void ResetWarning(object? sender, KeyPressEventArgs e)
        {
            if(sender != null && sender is TextBox textBox)
                textBox.BackColor = Color.White;
        }



        // METHOD NAME: OnGenerateQuoteClick
        // WRITTEN BY: Tyler J. Millershaski
        // DATE CREATED: 06 Sep 2026
        //
```

```
131         // METHOD PURPOSE:
132         //  Clear the previous quote, attempt to calculate the customer risk factor, and
133         //    generate the new quote output if the measurement values can be parsed.
134         //
135         // PARAMETERS LIST (in Parameter Order):
136         //  sender (object?) - The control that raised the click event (not used)
137         //  e (EventArgs) - Click event information (not used)
138         //
139         // RETURNS:
140         //  (Nothing; void.)
141         //
142         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
143         //  costPerCoverage (double) - The multiplier per dollar of requested policy coverage.
144         //  riskFactor (double) - The risk factor returned through TryCalculateRiskFactor.
145         //
146         // MODIFICATION HISTORY:
147         // WHO          WHEN            WHAT
148         // Millershaski 06 Sep 2026     Initial Version
149         void OnGenerateQuoteClick(object? sender, EventArgs e)
150         {
151             ClearAllOutput(); // This method is called before generating a new quote to ensure that previous output is not incorrectly associated with the new quote.
152             lblTotalAnnualPremium.Text = "Invalid Data"; // default to an error message in case the coverage or discount calculations fail
153
154             if(TryCalculateRiskFactor(out double riskFactor) == true)
155             {
156                 double costPerCoverage = CalculateCostPerCoverage (riskFactor);
157                 RefreshOutput(riskFactor, costPerCoverage);
158             }
159         }
160
161
162
163         // METHOD NAME: ClearAllOutput
164         // WRITTEN BY: Tyler J. Millershaski
165         // DATE CREATED: 06 Sep 2026
166         //
167         // METHOD PURPOSE:
168         //  Replace every quote output value with a dash so that previous quote details are not
```

```
169         //    displayed as part of the current quote. Input text boxes
              are not cleared.
170         //
171         // PARAMETERS LIST (in Parameter Order):
172         //  (None)
173         //
174         // RETURNS:
175         //  (Nothing; void.)
176         //
177         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
178         //  clearString (string) - The dash used as the placeholder for
              all cleared output labels.
179         //
180         // MODIFICATION HISTORY:
181         // WHO          WHEN            WHAT
182         // Millershaski 06 Sep 2026     Initial Version
183         void ClearAllOutput()
184         {
185             string clearString = "-"; // A non-empty clear string tends
                  to be better than an empty one.
186
187             lblCustomerName.Text = clearString;
188             lblRiskFactor.Text = clearString;
189             lblRiskCategory.Text = clearString;
190             lblCostPerThousand.Text = clearString;
191             lblInitialAnnualPremium.Text = clearString;
192             lblDiscountAmount.Text = clearString;
193             lblPremiumAfterDiscount.Text = clearString;
194             lblSalesTax.Text = clearString;
195             lblTotalAnnualPremium.Text = clearString;
196             lblCoverageAmount.Text = clearString;
197         }
198
199
200
201         // METHOD NAME: TryCalculateRiskFactor
202         // WRITTEN BY: Tyler J. Millershaski
203         // DATE CREATED: 06 Sep 2026
204         //
205         // METHOD PURPOSE:
206         //  Parse the age, height, and weight inputs, then calculate the
              risk factor.
207         //    (Highlights the first input TextBox that cannot be parsed.)
208         //
209         // PARAMETERS LIST (in Parameter Order):
210         //  riskFactor (out double) - Stores the calculated risk factor
              upon success. Set to zero if any input parsing fails.
211         //
212         // RETURNS:
```

```
213         //  bool - False when any input cannot be parsed. True otherwise.
214         //
215         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
216         //  age (double) - The customer age in years, parsed from txtAge.
217         //  denominator (double) - The bottom portion of the provided formula.
218         //  height (double) - The customer height in inches, parsed from txtHeight.
219         //  numerator (double) - The top portion of the provided formula.
220         //  weight (double) - The customer weight in pounds, parsed from txtWeight.
221         //
222         // MODIFICATION HISTORY:
223         // WHO          WHEN            WHAT
224         // Millershaski 06 Sep 2026     Initial Version
225         bool TryCalculateRiskFactor(out double riskFactor)
226         {
227             riskFactor = 0;
228
229             if(TryGetDoubleFromTextBox_DisplayError(txtAge, out double age, false) == false)
230                 return false;
231             if(TryGetDoubleFromTextBox_DisplayError(txtHeight, out double height, false) == false || height <= 0)
232                 return false;
233             if(TryGetDoubleFromTextBox_DisplayError(txtWeight, out double weight, false) == false)
234                 return false;
235
236             // note that the following formula was provided per the specifications
237             double numerator = age + Math.Sqrt((height * height) + (age * weight));
238             double denominator = weight - (4.01d * age);
239
240             if(denominator == 0) // prevent division by zero
241                 return false;
242
243             riskFactor = numerator / denominator;
244             return true;
245         }
246
247
248
249         // METHOD NAME: TryGetDoubleFromTextBox_DisplayError
250         // WRITTEN BY: Tyler J. Millershaski
251         // DATE CREATED: 06 Sep 2026
252         //
253         // METHOD PURPOSE:
```

```
254         //  Attempt to parse the specified text box as a double and highlight that text box upon fail
255         //
256         // PARAMETERS LIST (in Parameter Order):
257         //  allowNegative (bool) - True to accept negative values. False to reject them.
258         //  someTextBox (TextBox) - The input control whose Text property will be parsed.
259         //  value (out double) - Stores the parsed value on success. Set to 0 upon fail
260         //
261         // RETURNS:
262         //  bool - True if parsing succeeds and the value is within the allowed range. False otherwise.
263         //
264         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
265         //  (None)
266         //
267         // MODIFICATION HISTORY:
268         // WHO          WHEN            WHAT
269         // Millershaski 06 Sep 2026     Initial Version
270         bool TryGetDoubleFromTextBox_DisplayError(TextBox someTextBox, out double value, bool allowNegative)
271         {
272             if(TryGetDoubleFromTextBox(someTextBox, out value) == true)
273             {
274                 if(allowNegative == true || value > 0)
275                     return true;
276             }
277             DisplayInputError(someTextBox);
278             return false;
279         }
280
281
282
283         // METHOD NAME: DisplayInputError
284         // WRITTEN BY: Tyler J. Millershaski
285         // DATE CREATED: 06 Sep 2026
286         //
287         // METHOD PURPOSE:
288         //  Mark an input text box with a light-pink background and request keyboard focus so
289         //    the user can correct its contents.
290         //
291         // PARAMETERS LIST (in Parameter Order):
292         //  someTextBox (TextBox) - The input control to highlight and focus.
293         //
294         // RETURNS:
```

```
295         //  (Nothing; void.)
296         //
297         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
298         //  (None)
299         //
300         // NOTES:
301         //  A text box subscribed to ResetWarning can restore its white
              background on the next
302         //    KeyPress event.
303         //
304         // MODIFICATION HISTORY:
305         // WHO          WHEN            WHAT
306         // Millershaski 06 Sep 2026     Initial Version
307         void DisplayInputError(TextBox someTextBox)
308         {
309             someTextBox.BackColor = Color.LightPink;
310             someTextBox.Focus();
311         }
312
313
314
315         // METHOD NAME: TryGetDoubleFromTextBox
316         // WRITTEN BY: Tyler J. Millershaski
317         // DATE CREATED: 06 Sep 2026
318         //
319         // METHOD PURPOSE:
320         //  Attempt to convert the text box contents to a double (default
              culture).
321         //
322         // PARAMETERS LIST (in Parameter Order):
323         //  someTextBox (TextBox) - The TextBox containing the numeric
              text.
324         //  value (out double) - Stores the parsed number on success.
              Stores zero upon fail.
325         //
326         // RETURNS:
327         //  bool - The success or failure result returned by
              double.TryParse.
328         //
329         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
330         //  (None)
331         //
332         // MODIFICATION HISTORY:
333         // WHO          WHEN            WHAT
334         // Millershaski 06 Sep 2026     Initial Version
335         bool TryGetDoubleFromTextBox(TextBox someTextBox, out double
              value)
336         {
337             return double.TryParse(someTextBox.Text, out value);
```

```
338         }
339
340
341
342         // METHOD NAME: CalculateCostPerCoverage
343         // WRITTEN BY: Tyler J. Millershaski
344         // DATE CREATED: 06 Sep 2026
345         //
346         // METHOD PURPOSE:
347         //  Calculates the per-dollar coverage multiplier based on the
              normalized risk factor. The formula is (10.1 - absolute
              normalized risk factor) / 10.
348         //
349         // PARAMETERS LIST (in Parameter Order):
350         //  riskFactor (double) - The previously calculated risk factor.
351         //
352         // RETURNS:
353         //  double - The per-dollar coverage multiplier
354         //
355         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
356         //  (None)
357         //
358         // MODIFICATION HISTORY:
359         // WHO          WHEN            WHAT
360         // Millershaski 06 Sep 2026     Initial Version
361         double CalculateCostPerCoverage(double riskFactor)
362         {
363             if(Math.Abs(riskFactor) > 10.0) // note that a riskFactor of
                  Abs(10.0) will not be divided
364             {
365                 // Divide until only 1 digit remains (per specification)
366                 while(Math.Abs(riskFactor) >= 10)
367                 {
368                     riskFactor /= 10;
369                 }
370             }
371
372             return (10.1 - Math.Abs(riskFactor)) / 10d;
373         }
374
375
376
377         // METHOD NAME: RefreshOutput
378         // WRITTEN BY: Tyler J. Millershaski
379         // DATE CREATED: 06 Sep 2026
380         //
381         // METHOD PURPOSE:
382         //  Display the customer details and calculated quote amounts.
              Stop if any coverage, premium, or discount refresh fails.
```

```
383         //
384         // PARAMETERS LIST (in Parameter Order):
385         //  riskFactor (double) - The original calculated risk factor to
              display.
386         //  costPerCoverage (double) - The multiplier per dollar of
              policy coverage.
387         //
388         // RETURNS:
389         //  (Nothing; void.)
390         //
391         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
392         //  annualPremium (double) - The annual premium before discounts
              and sales tax.
393         //  coverageAmount (double) - The requested policy coverage
              amount.
394         //  discountAmount (double) - The selected discount in dollars.
395         //  premiumAfterDiscount (double) - The annual premium minus the
              discount amount.
396         //  salesTaxAmount (double) - The sales tax on the premium after
              discount.
397         //  totalAnnualPremium (double) - The premium after discount plus
              sales tax.
398         //
399         // MODIFICATION HISTORY:
400         // WHO          WHEN            WHAT
401         // Millershaski 06 Sep 2026     Initial Version
402         void RefreshOutput(double riskFactor, double costPerCoverage)
403         {
404             lblCustomerName.Text = GetCustomerName();
405             lblRiskFactor.Text = riskFactor.ToString("F2");
406             lblRiskCategory.Text = GetRiskCategoryLabel(riskFactor);
407             lblCostPerThousand.Text = (costPerCoverage * 1000).ToString
                  ("C2"); // note that it's displayed to the user as "per
                  1000" so we multiply by 1000 to get the correct value to
                  display
408
409             if(TryRefreshCoverageAmount(out double coverageAmount) ==
                  false)
410                 return;
411
412             RefreshInitialAnnualPremium(coverageAmount, costPerCoverage,
                  out double annualPremium);
413             if(TryRefreshDiscountAmount(annualPremium, out double
                  discountAmount) == false)
414                 return;
415
416             double premiumAfterDiscount = annualPremium - discountAmount;
417             lblPremiumAfterDiscount.Text = premiumAfterDiscount.ToString
                  ("C2");
```

```
418
419             double salesTaxAmount = premiumAfterDiscount * SALES_TAX_RATE;
420             lblSalesTax.Text = salesTaxAmount.ToString("C2");
421
422             double totalAnnualPremium = premiumAfterDiscount +
                  salesTaxAmount;
423             lblTotalAnnualPremium.Text = totalAnnualPremium.ToString
                  ("C2");
424         }
425
426
427
428         // METHOD NAME: GetCustomerName
429         // WRITTEN BY: Tyler J. Millershaski
430         // DATE CREATED: 06 Sep 2026
431         //
432         // METHOD PURPOSE:
433         //  Returns the full customer name from the first name, middle
              initial, and last name.
434         //
435         // PARAMETERS LIST (in Parameter Order):
436         //  (None)
437         //
438         // RETURNS:
439         //  string - "(No Name Provided)" if the first and last names are
              blank. Otherwise, the full name.
440         //
441         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
442         //  fullName (string) - The combined customer name before the
              final trim.
443         //
444         // MODIFICATION HISTORY:
445         // WHO          WHEN            WHAT
446         // Millershaski 06 Sep 2026     Initial Version
447         string GetCustomerName()
448         {
449             if(String.IsNullOrWhiteSpace(txtFirstName.Text) &&
                  String.IsNullOrWhiteSpace(txtLastName.Text)) // if both
                  last name and first name are blank, return a default string
                  (even if middle initial is populated)
450                 return "(No Name Provided)";
451
452             string fullName = "";
453             if(String.IsNullOrWhiteSpace(txtFirstName.Text) == false)
454                 fullName += txtFirstName.Text.Trim();
455             if(String.IsNullOrWhiteSpace(txtMiddleInitial.Text) == false)
456                 fullName += " " + txtMiddleInitial.Text.Trim();
457             if(String.IsNullOrWhiteSpace(txtLastName.Text) == false)
458                 fullName += " " + txtLastName.Text.Trim();
```

```
459
460             return fullName.Trim(); // trim to remove any leading or trailing whitespace in case either name is null or has leading/trailing whitespace
461         }
462
463
464
465         // METHOD NAME: GetRiskCategoryLabel
466         // WRITTEN BY: Tyler J. Millershaski
467         // DATE CREATED: 06 Sep 2026
468         //
469         // METHOD PURPOSE:
470         //  Get the "safe" or "unsafe label for the calculated risk factor.
471         //
472         // PARAMETERS LIST (in Parameter Order):
473         //  riskFactor (double) - The calculated risk factor.
474         //
475         // RETURNS:
476         //  string - "Safe" if riskFactor is greater than or equal to zero. "Unsafe" otherwise.
477         //
478         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
479         //  (None)
480         //
481         // MODIFICATION HISTORY:
482         // WHO          WHEN            WHAT
483         // Millershaski 06 Sep 2026     Initial Version
484         string GetRiskCategoryLabel(double riskFactor)
485         {
486             if(riskFactor >= 0)
487                 return "Safe";
488             else
489                 return "Unsafe";
490         }
491
492
493
494         // METHOD NAME: TryRefreshCoverageAmount
495         // WRITTEN BY: Tyler J. Millershaski
496         // DATE CREATED: 06 Sep 2026
497         //
498         // METHOD PURPOSE:
499         //  Attempt to read the policy coverage amount and display it as currency.
500         //    Displays an error message upon fail.
501         //
502         // PARAMETERS LIST (in Parameter Order):
```

```
503         //  coverageAmount (out double) - Stores the coverage amount on success. Set to zero upon fail.
504         //
505         // RETURNS:
506         //  bool - True if the coverage input is accepted. False otherwise.
507         //
508         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
509         //  (None)
510         //
511         // MODIFICATION HISTORY:
512         // WHO          WHEN            WHAT
513         // Millershaski 06 Sep 2026     Initial Version
514         bool TryRefreshCoverageAmount(out double coverageAmount)
515         {
516             if(TryGetCoverageAmount(out coverageAmount) == true)
517             {
518                 lblCoverageAmount.Text = coverageAmount.ToString("C2");
519                 return true;
520             }
521             else
522             {
523                 lblCoverageAmount.Text = "Invalid Coverage Amount";
524                 return false;
525             }
526         }
527
528
529
530         // METHOD NAME: TryRefreshInitialAnnualPremium
531         // WRITTEN BY: Tyler J. Millershaski
532         // DATE CREATED: 06 Sep 2026
533         //
534         // METHOD PURPOSE:
535         //  Calculates the initial annual premium and display it as currency.
536         //
537         // PARAMETERS LIST (in Parameter Order):
538         //  coverageAmount (double) - The requested policy coverage amount.
539         //  costPerCoverage (double) - The multiplier per dollar of policy coverage.
540         //  annualPremium (out double) - Stores coverageAmount multiplied by costPerCoverage.
541         //
542         // RETURNS:
543         //  (Nothing; void.)
544         //
545         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
```

```
546         //  (None)
547         //
548         // MODIFICATION HISTORY:
549         // WHO          WHEN            WHAT
550         // Millershaski 06 Sep 2026     Initial Version
551         void RefreshInitialAnnualPremium(double coverageAmount, double
              costPerCoverage, out double annualPremium)
552         {
553             annualPremium = coverageAmount * costPerCoverage;
554             lblInitialAnnualPremium.Text = annualPremium.ToString("C2");
555         }
556
557
558
559         // METHOD NAME: TryRefreshDiscountAmount
560         // WRITTEN BY: Tyler J. Millershaski
561         // DATE CREATED: 06 Sep 2026
562         //
563         // METHOD PURPOSE:
564         //  Attempt to calculate the selected discount and display it as
              currency.
565         //    Display an error message upon fail.
566         //
567         // PARAMETERS LIST (in Parameter Order):
568         //  annualPremium (double) - The annual premium before discounts
              and sales tax.
569         //  discountAmount (out double) - Stores the discount in dollars.
              Set to zero for no discount or upon fail.
570         //
571         // RETURNS:
572         //  bool - True if the discount calculation succeeds or no
              discount is selected. False otherwise.
573         //
574         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
575         //  (None)
576         //
577         // MODIFICATION HISTORY:
578         // WHO          WHEN            WHAT
579         // Millershaski 06 Sep 2026     Initial Version
580         bool TryRefreshDiscountAmount(double annualPremium, out double
              discountAmount)
581         {
582             if(TryCalculateDiscountAmount(annualPremium, out
                  discountAmount) == true)
583             {
584                 lblDiscountAmount.Text = discountAmount.ToString("C2");
585                 return true;
586             }
587             else
```

```
588             {
589                 lblDiscountAmount.Text = "Invalid Discount Amount";
590                 return false;
591             }
592         }
593 
594 
595 
596         // METHOD NAME: TryGetCoverageAmount
597         // WRITTEN BY: Tyler J. Millershaski
598         // DATE CREATED: 06 Sep 2026
599         //
600         // METHOD PURPOSE:
601         //  Attempt to parse the policy coverage amount.
602         //    Reject zero or negative values and highlight the TextBox
              upon fail.
603         //
604         // PARAMETERS LIST (in Parameter Order):
605         //  coverageAmount (out double) - Stores the parsed coverage
              amount on success. Set to zero upon fail.
606         //
607         // RETURNS:
608         //  bool - False if parsing fails or the coverage amount is less
              than or equal to zero. True otherwise.
609         //
610         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
611         //  (None)
612         //
613         // MODIFICATION HISTORY:
614         // WHO          WHEN            WHAT
615         // Millershaski 06 Sep 2026     Initial Version
616         bool TryGetCoverageAmount(out double coverageAmount)
617         {
618             if(TryGetDoubleFromTextBox(txtCoverageAmount, out
                  coverageAmount) == false || coverageAmount <= 0) // note
                  that the 0 fails and not just negative values
619             {
620                 coverageAmount = 0;
621                 DisplayInputError(txtCoverageAmount);
622                 return false;
623             }
624             return true;
625         }
626 
627 
628 
629         // METHOD NAME: TryCalculateDiscountAmount
630         // WRITTEN BY: Tyler J. Millershaski
631         // DATE CREATED: 06 Sep 2026
```

```
632         //
633         // METHOD PURPOSE:
634         //  Calculate the percentage or flat dollar discount selected by the radio buttons.
635         //    Use zero when no discount is selected.
636         //
637         // PARAMETERS LIST (in Parameter Order):
638         //  annualPremium (double) - The annual premium before discounts and sales tax.
639         //  discountAmount (out double) - Stores the discount in dollars. Set to zero for no discount or upon fail.
640         //
641         // RETURNS:
642         //  bool - False if the selected discount input is rejected. True otherwise.
643         //
644         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
645         //  discountType (int) - The selected discount: 0 = none, 1 = percentage, 2 = flat amount.
646         //
647         // MODIFICATION HISTORY:
648         // WHO          WHEN            WHAT
649         // Millershaski 06 Sep 2026     Initial Version
650         bool TryCalculateDiscountAmount(double annualPremium, out double discountAmount)
651         {
652             discountAmount = 0;
653 
654             int discountType = GetDiscountType();
655             if(discountType == 1) // percentage
656                 return TryGetPercentageDiscountAmount(annualPremium, out discountAmount);
657             else if(discountType == 2) // flat amount
658                 return TryGetFlatDiscountAmount(annualPremium, out discountAmount);
659 
660             return true;
661         }
662 
663 
664 
665         // METHOD NAME: GetDiscountType
666         // WRITTEN BY: Tyler J. Millershaski
667         // DATE CREATED: 06 Sep 2026
668         //
669         // METHOD PURPOSE:
670         //  Get the selected discount type from the radio buttons and converts it to an integer.
671         //
```

```
672         // PARAMETERS LIST (in Parameter Order):
673         //  (None)
674         //
675         // RETURNS:
676         //  int - 1 for a percentage discount, 2 for a flat discount, or
              0 for no discount.
677         //
678         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
679         //  (None)
680         //
681         // MODIFICATION HISTORY:
682         // WHO          WHEN            WHAT
683         // Millershaski 06 Sep 2026     Initial Version
684         int GetDiscountType()
685         {
686             if(rdoPercentageDiscount.Checked == true)
687                 return 1;
688             else if(rdoFlatDiscount.Checked == true)
689                 return 2;
690             else
691                 return 0; // default to no discount if none are selected
692         }
693
694
695
696         // METHOD NAME: TryGetPercentageDiscountAmount
697         // WRITTEN BY: Tyler J. Millershaski
698         // DATE CREATED: 06 Sep 2026
699         //
700         // METHOD PURPOSE:
701         //  Attempt to parse the percentage discount and calculate its
              dollar amount.
702         //    Reject percentages below 0 or above 100 and highlight the
              TextBox upon fail.
703         //
704         // PARAMETERS LIST (in Parameter Order):
705         //  annualPremium (double) - The annual premium before discounts
              and sales tax.
706         //  discountAmount (out double) - Stores the calculated dollar
              discount on success. Set to zero upon fail.
707         //
708         // RETURNS:
709         //  bool - False if parsing fails or the percentage is below 0 or
              above 100. True otherwise.
710         //
711         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
712         //  percentageDiscount (double) - The entered discount percentage
              (5 means 5%).
713         //
```

```
714         // MODIFICATION HISTORY:
715         // WHO          WHEN            WHAT
716         // Millershaski 06 Sep 2026     Initial Version
717         bool TryGetPercentageDiscountAmount(double annualPremium, out
              double discountAmount)
718         {
719             discountAmount = 0;
720             if(TryGetDoubleFromTextBox(txtPercentageDiscount, out double
                  percentageDiscount) == false || percentageDiscount < 0 ||
                  percentageDiscount > 100)
721             {
722                 DisplayInputError(txtPercentageDiscount);
723                 return false;
724             }
725
726             discountAmount = annualPremium * (percentageDiscount / 100d);
727             return true;
728         }
729
730
731
732         // METHOD NAME: TryGetFlatDiscountAmount
733         // WRITTEN BY: Tyler J. Millershaski
734         // DATE CREATED: 06 Sep 2026
735         //
736         // METHOD PURPOSE:
737         //  Attempt to parse the flat dollar discount. Reject negative
              values and highlight the TextBox upon fail.
738         //
739         // PARAMETERS LIST (in Parameter Order):
740         //  annualPremium (double) - The annual premium before discounts
              and sales tax, also used as the maximum discount.
741         //  discountAmount (out double) - Stores the smaller of the
              entered discount and the annual premium. Set to zero upon fail.
742         //
743         // RETURNS:
744         //  bool - False if parsing fails or the flat discount is below
              zero. True otherwise.
745         //
746         // LOCAL VARIABLE DICTIONARY (in Alphabetical Order):
747         //  flatDiscount (double) - The entered flat dollar discount
              before applying the premium limit.
748         //
749         // MODIFICATION HISTORY:
750         // WHO          WHEN            WHAT
751         // Millershaski 06 Sep 2026     Initial Version
752         bool TryGetFlatDiscountAmount(double annualPremium, out double
              discountAmount)
753         {
```

```
754             discountAmount = 0;
755             if(TryGetDoubleFromTextBox(txtFlatDiscount, out double
                  flatDiscount) == false || flatDiscount < 0)
756             {
757                 DisplayInputError(txtFlatDiscount);
758                 return false;
759             }
760 
761             discountAmount = Math.Min(flatDiscount, annualPremium); //
                  ensures that the discount amount does not exceed the annual
                  premium
762             return true;
763         }
764     }
765 }
766 
```